Intro2SE - Requirements Analysis - 4

PathToGrad

The student team is required to complete the Software Requirements Specification (SRS) document for the assigned course project, following the attached template.

Software Engineering Department
Faculty of Information and Technology
VNU-HCMUS

Table of Contents

Software Requirements Specification

Objectives

This document focus on the following topics:

- ✔ Completing the Software Requirements Specification (SRS) document with the following contents:
    - Elaborate on the Problem Statement
    - Overview of Requirements (Functional and Non-Functional), Stakeholders
    - Use Case Model
    - Use Case Specifications
    - Create Prototype and Mockup Diagrams of the System Interface
- ✔ Understanding the Project Requirement Analysis document.
- ✔ This document will be used as input for AI tools to verify the quality of subsequent project artifacts.

All project artifacts must remain consistent and synchronized.

For example, if the project proposal is modified during this phase, a new version of the proposal must be documented.

By the conclusion of the project, all versions of every artifact must be submitted to demonstrate the evolution of your work.

1 Member Contribution Assessment

24127427 - Trần Nhựt Đăng Khoa - 20%

- **Task Evidence:**

Task 1: Sprint Planning & Jira Management

Description: Organized the "Final Review" sprint, assigned quality assurance tasks, and tracked peer-review progress across the team to ensure all deadlines were met and workloads were balanced.

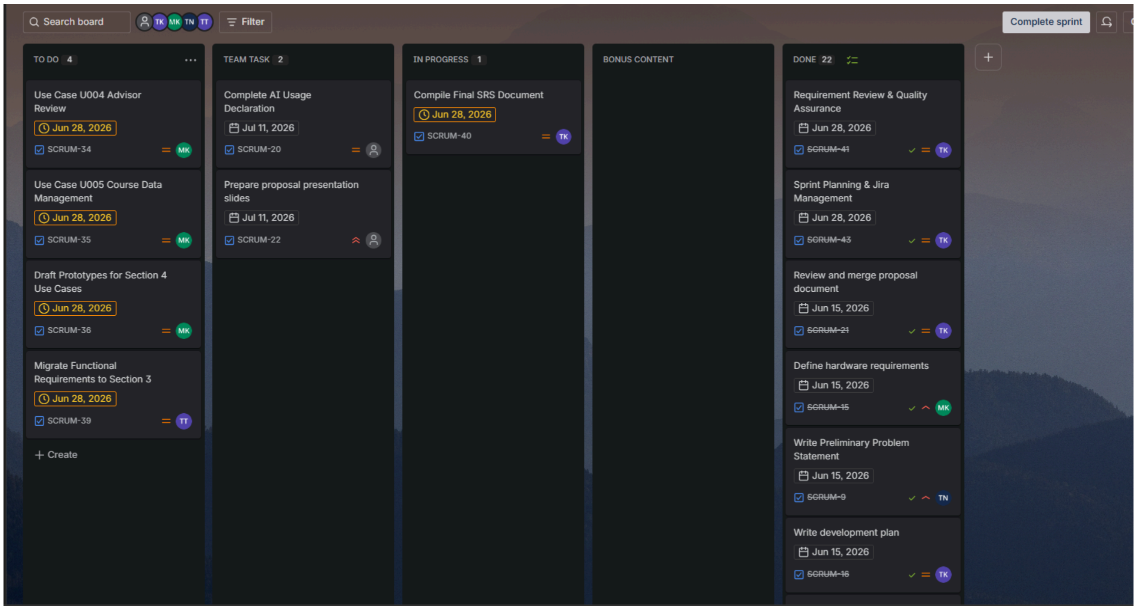
Task 2: GitHub Repository Structure & Documentation Setup

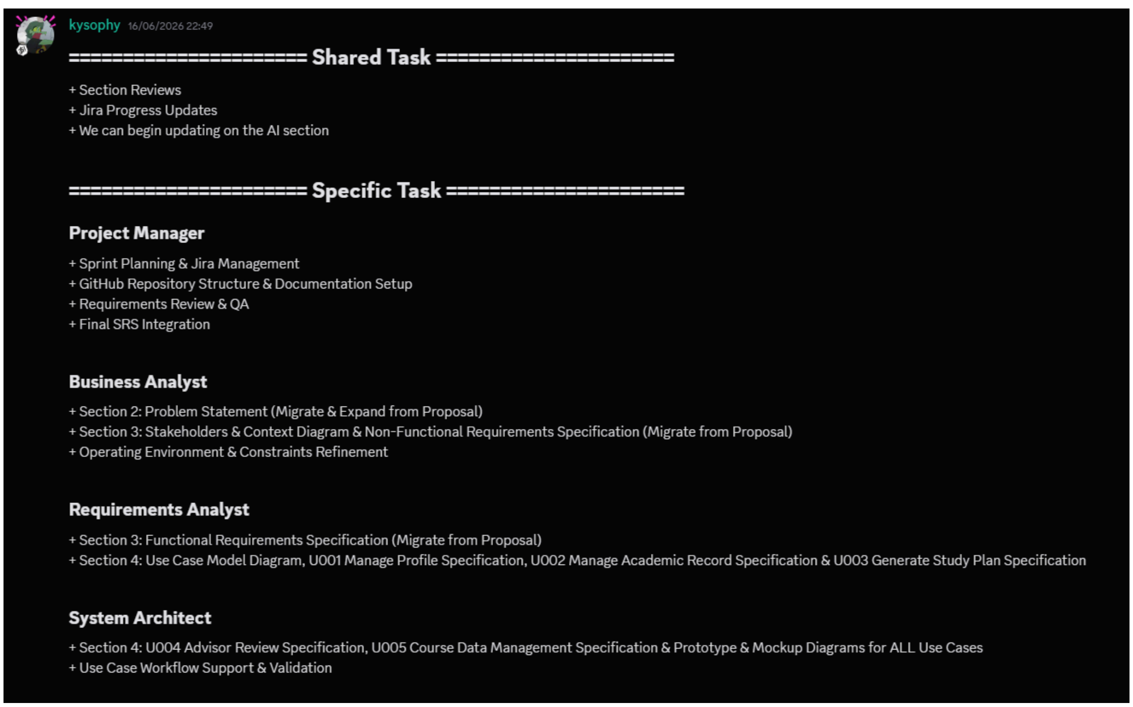
Description: Maintained the `/docs/requirements` folder structure on GitHub, enforced markdown documentation migration from the team, and centralized the system's architectural and prototype images for the final report.

Task 3: Requirements Review & QA

Description: Conducted comprehensive quality assurance across all drafted sections. Verified consistency between the Use Case Model and Problem Statement, and ensured all 17 Functional Requirements and 9 Non-Functional Requirements were accurately integrated without contradictions.

- **Visual Evidence:**





kysophy 16/06/2026 22:49

==================== Shared Task ====================

+ Section Reviews
+ Jira Progress Updates
+ We can begin updating on the AI section

==================== Specific Task ====================

Project Manager

+ Sprint Planning & Jira Management
+ GitHub Repository Structure & Documentation Setup
+ Requirements Review & QA
+ Final SRS Integration

Business Analyst

+ Section 2: Problem Statement (Migrate & Expand from Proposal)
+ Section 3: Stakeholders & Context Diagram & Non-Functional Requirements Specification (Migrate from Proposal)
+ Operating Environment & Constraints Refinement

Requirements Analyst

+ Section 3: Functional Requirements Specification (Migrate from Proposal)
+ Section 4: Use Case Model Diagram, U001 Manage Profile Specification, U002 Manage Academic Record Specification & U003 Generate Study Plan Specification

System Architect

+ Section 4: U004 Advisor Review Specification, U005 Course Data Management Specification & Prototype & Mockup Diagrams for ALL Use Cases
+ Use Case Workflow Support & Validation

===================== **Deliverables** =====================

Project Manager

+ Final SRS Document
+ Jira Tracking
+ QA Review Report

Business Analyst

+ Section 2: Problem Statement
+ Section 3: Stakeholders & Context Diagram & NFR Specification

Requirements Analyst

+ Section 3: Functional Requirements
+ Section 4: Use Case Diagram & U001-U003 Specifications

System Architect

+ Section 4: U004-U005 Specifications & Prototypes and Mockups for Use Cases

===================== **Github** =====================

- i will create and organize all required folders, placeholder files, and document templates inside /docs/requirements
- After completing your assigned sections, fill in the corresponding files
- for diagram I can suggest using Draw.io / Miro, but feel free to use anything else that is more comfortable to you

```
requirements
  problem_statement
    problem_statement.md
  requirements_analysis
    use_cases
      U001_manage_profile.md
      U002_manage_academic_re...
      U003_generate_study_plan....
      U004_advisor_review.md
      U005_course_data_manage...
    use_case_diagram.md
  requirements_overview
  README.md
  features.md
  user-stories.md
```

24127543 - Nguyễn Trương Ngọc Thảo - 20%

- **Task Evidence:**

Task 1: Format Section 2: Problem Statement

Description: Format and refine section 2 from existing document.

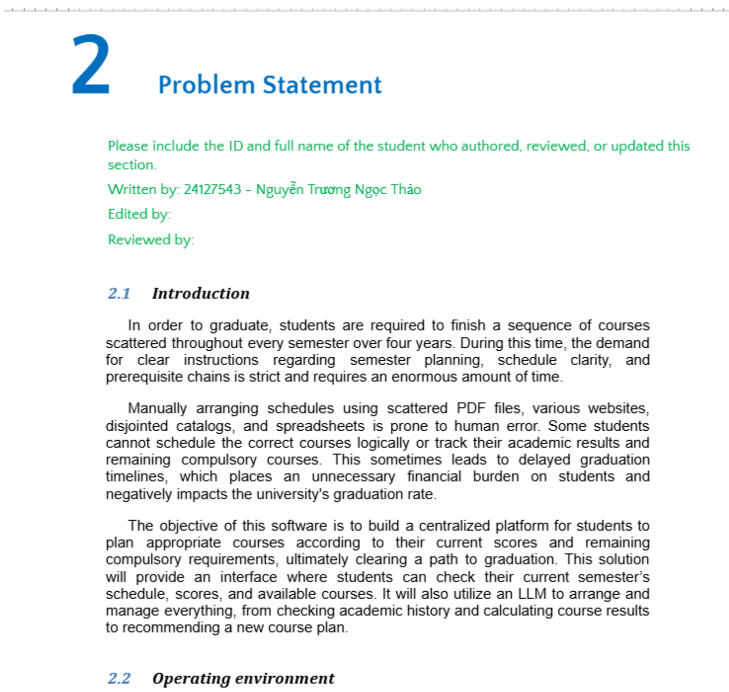
Task 2: Draft Context Diagram for Section 3

Description: Draw context diagram for system.

Task 3: Migrate Non-Functional Requirements to Section 3

Description: Migrate Non-Functional Requirements from existing document to current document.

- **Visual Evidence:**



2 Problem Statement

Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by: 24127543 - Nguyễn Trương Ngọc Thảo

Edited by:

Reviewed by:

2.1 *Introduction*

In order to graduate, students are required to finish a sequence of courses scattered throughout every semester over four years. During this time, the demand for clear instructions regarding semester planning, schedule clarity, and prerequisite chains is strict and requires an enormous amount of time.

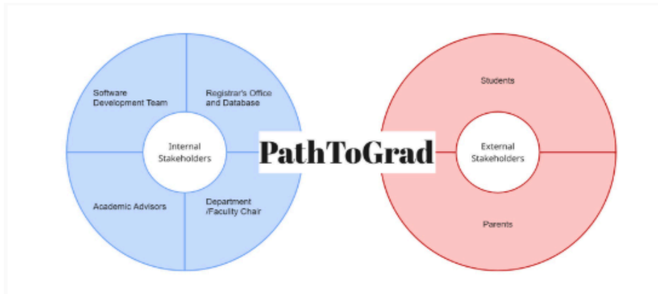
Manually arranging schedules using scattered PDF files, various websites, disjointed catalogs, and spreadsheets is prone to human error. Some students cannot schedule the correct courses logically or track their academic results and remaining compulsory courses. This sometimes leads to delayed graduation timelines, which places an unnecessary financial burden on students and negatively impacts the university's graduation rate.

The objective of this software is to build a centralized platform for students to plan appropriate courses according to their current scores and remaining compulsory requirements, ultimately clearing a path to graduation. This solution will provide an interface where students can check their current semester's schedule, scores, and available courses. It will also utilize an LLM to arrange and manage everything, from checking academic history and calculating course results to recommending a new course plan.

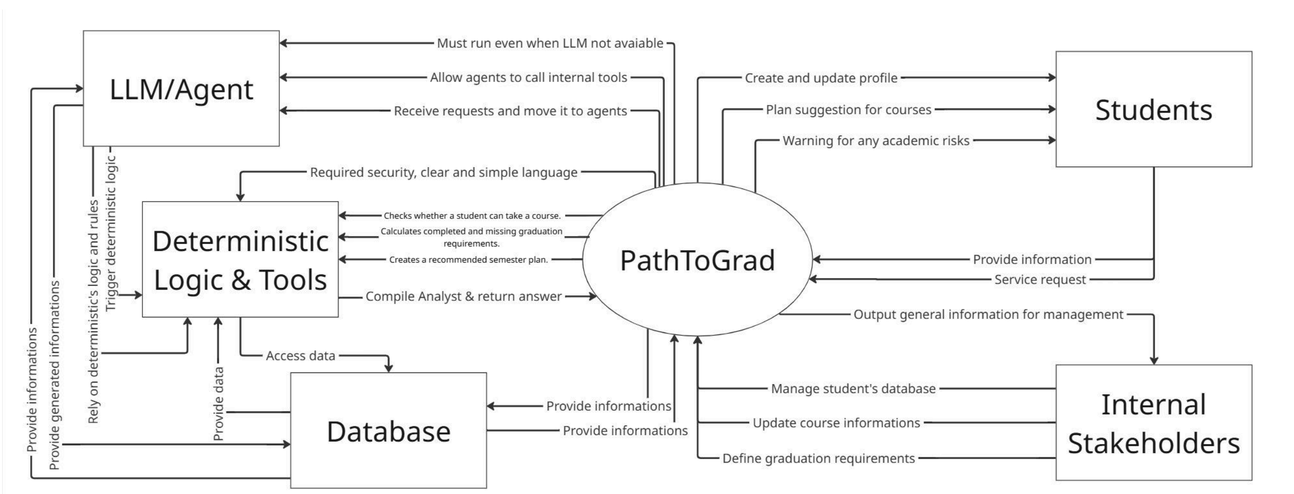
2.2 *Operating environment*

3.1 *Stakeholders*

Written by: 24127543 - Nguyễn Trương Ngọc Thảo
Edited by:
Reviewed by:



No.	Stakeholder	Description
1	Software Development Team	Our development team works on the software and in the current version holds the data generating job.
2	Registrar's Office and Database	On future developments, the Registrar's Office holds the academic records and defines graduation rules.
3	Academic Advisors	The Academic Advisors can see the current class list, review student plans, records and identify possible conflict risks. Through this access, they can guide the student more precisely.



24127563 - Trần Thị Ngọc Trâm - 30%

- **Task Evidence:**

Task 1: Write Functional Requirements Specification

Description: Created Section 3.2.1, defined 17 functional requirements and grouped them into Student Data Management, LLM Planning Agent and Tool Interface, Course Data and Academic Checking, Study Plan Generation and Explanation, and Advisor/Admin/Traceability.

Task 2: Create Use Case Diagram

Description: Created the use case diagram for the PathToGrad system. Identified the main actors, including Student, Academic Advisor, Academic Staff/Admin, and LLM Provider.

Task 3: Write U001–U003 Use Case Specifications

Description: Wrote the use case specifications for U001 Manage Profile, U002 Manage Academic Record, and U003 Generate Study Plan. Defined the actor, pre-condition, result, main scenario, alternative scenarios, and non-functional constraints for each use case.

Task 4: Upload Requirements to GitHub

Description: Uploaded and updated the requirements analysis files on GitHub, including the functional requirements, use case diagram, and U001–U003 use case specification files.

- **Visual Evidence:**

3.2 Requirements

3.2.1. Functional Requirements Specification

Written by: 24127563 - Trần Thị Ngọc Trâm

Edited by:

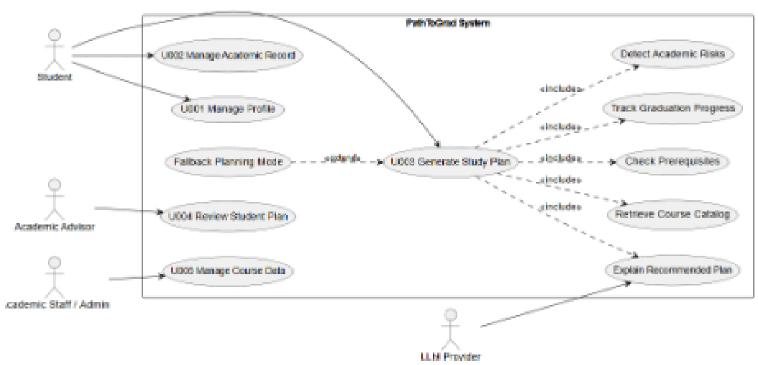
Reviewed by:

The functional requirements describe the main functions that PathToGrad must provide. These requirements are based on the proposal, the stakeholder needs, and the current system scope. The system focuses on academic planning, prerequisite checking, graduation progress tracking, and LLM-supported study plan generation.

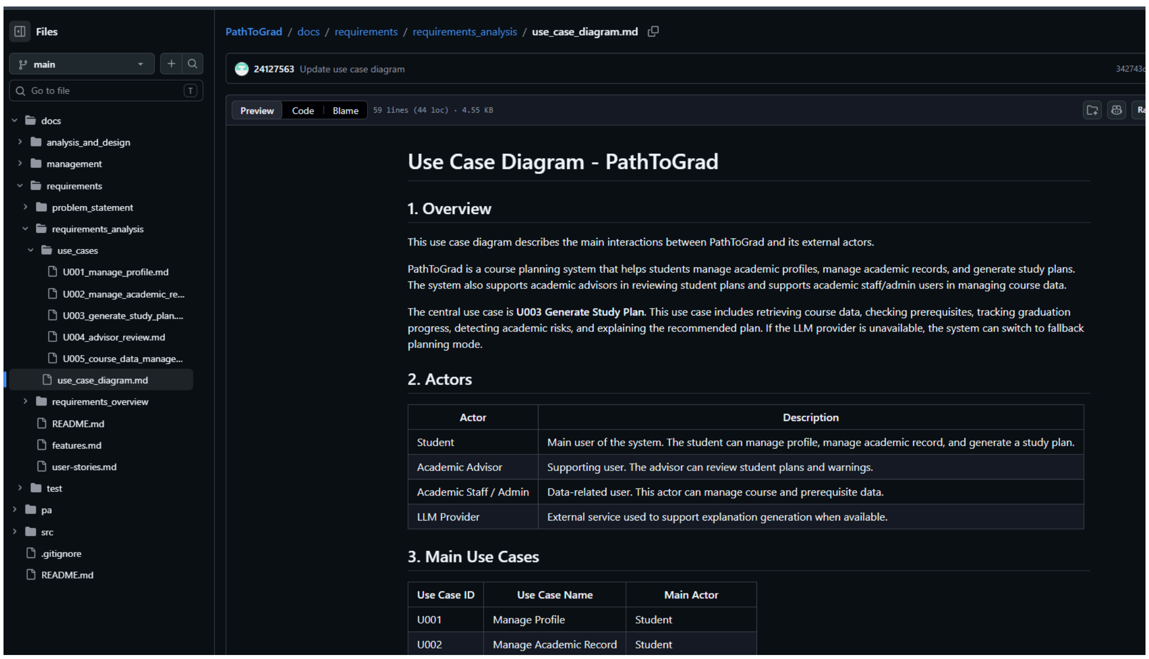
FR-S1: Student Data Management

ID	Functional Requirement
FR-01	The system shall allow students to create and update their academic profile.
FR-02	The system shall allow students to enter completed courses, failed courses, grades, and credits.

The use case diagram describes the main interactions between PathToGrad and external actors. The Student can manage a profile, manage academic records, and generate a study plan. The Academic Advisor can review student plans, while Academic Staff/Admin can manage course data. The LLM Provider supports explanation generation when available. The central use case is U003 Generate Study Plan, which includes retrieving course data, checking prerequisites, tracking graduation progress, detecting academic risks, and explaining the recommended plan.



Use case ID	U001
Use Case	Manage Profile.
Brief Description	This use case allows students to create and update their academic profile. The profile includes major, intake year, current semester, and target credit load. The system uses this information for course planning.
Actor	Student.
Pre-Condition	The student has opened the PathToGrad system.
Result	The student profile is saved and can be used by the LLM Planning Agent and internal planning tools.
Main Scenario	1. The student opens the profile page. 2. The system displays the profile form. 3. The student enters major, intake year, current semester, and target credit load. 4. The student submits the form. 5. The system validates the input data. 6. The system saves the student profile. 7. The system shows a success message.
	Missing required Information: The student leaves a required field


Files

main

Go to file

- docs
  - analysis_and_design
  - management
  - requirements
    - problem_statement
    - requirements_analysis
      - use_cases
        - U001_manage_profile.md
        - U002_manage_academic_re...
        - U003_generate_study_plan....
        - U004_advisor_review.md
        - U005_course_data_manage...
      - use_case_diagram.md
    - requirements_overview
    - README.md
    - features.md
    - user-stories.md
  - test
- pa
- src
- .gitignore
- README.md

PathToGrad / docs / requirements / requirements_analysis / use_case_diagram.md

24127563 Update use case diagram 342743c

Preview Code Blame 59 lines (44 loc) · 4.55 KB

Use Case Diagram - PathToGrad

1. Overview

This use case diagram describes the main interactions between PathToGrad and its external actors.

PathToGrad is a course planning system that helps students manage academic profiles, manage academic records, and generate study plans. The system also supports academic advisors in reviewing student plans and supports academic staff/admin users in managing course data.

The central use case is **U003 Generate Study Plan**. This use case includes retrieving course data, checking prerequisites, tracking graduation progress, detecting academic risks, and explaining the recommended plan. If the LLM provider is unavailable, the system can switch to fallback planning mode.

2. Actors

Actor	Description
Student	Main user of the system. The student can manage profile, manage academic record, and generate a study plan.
Academic Advisor	Supporting user. The advisor can review student plans and warnings.
Academic Staff / Admin	Data-related user. This actor can manage course and prerequisite data.
LLM Provider	External service used to support explanation generation when available.

3. Main Use Cases

Use Case ID	Use Case Name	Main Actor
U001	Manage Profile	Student
U002	Manage Academic Record	Student

24127080 - Ông Khánh Minh - 30%

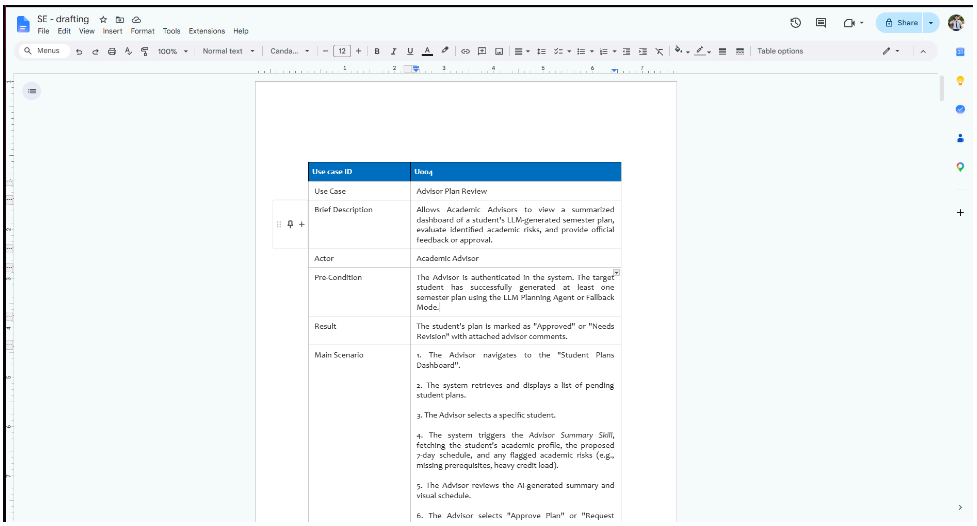
- **Task Evidence:**

Task 1: 4.2.4 - U004 Advisor Review Specification

Task 2: 4.2.5 - U005 Course Data Management Specification

Description: Declares and specifies specific use cases of U004 and U005 of the developing project with clear description of how interactions should behave in perfect condition and returning behaviour made by the product.

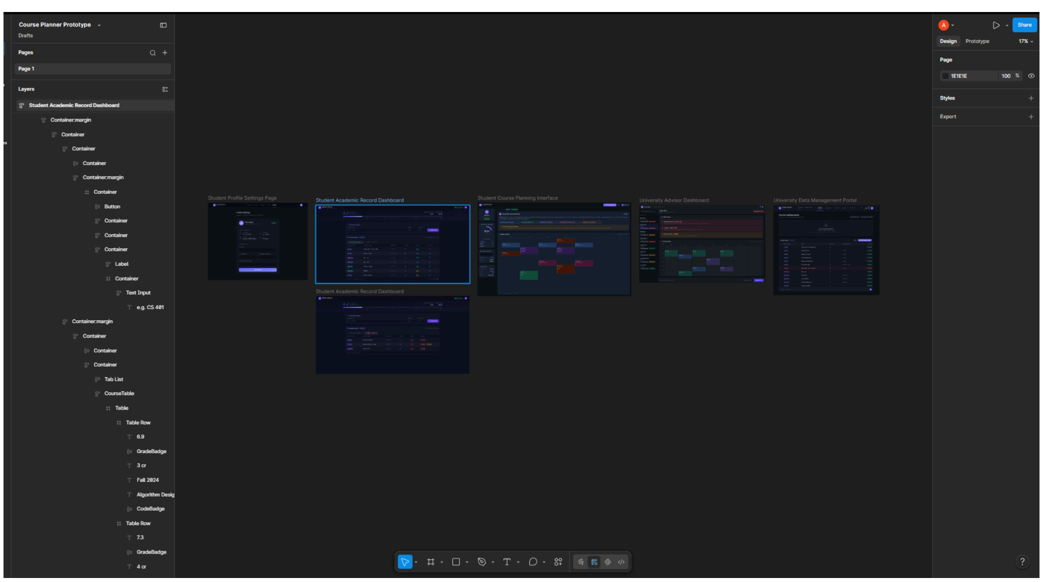
- **Visual Evidence:**



Use case ID	U004
Use Case	Advisor Plan Review
Brief Description	Allows Academic Advisors to view a summarized dashboard of a student's LLM-generated semester plan, evaluate identified academic risks, and provide official feedback or approval.
Actor	Academic Advisor
Pre-Condition	The Advisor is authenticated in the system. The target student has successfully generated at least one semester plan using the LLM Planning Agent or Fallback Mode.
Result	The student's plan is marked as "Approved" or "Needs Revision" with attached advisor comments.
Main Scenario	1. The Advisor navigates to the "Student Plans Dashboard". 2. The system retrieves and displays a list of pending student plans. 3. The Advisor selects a specific student. 4. The system triggers the *Advisor Summary Skill*, fetching the student's academic profile, the proposed 7-day schedule, and any flagged academic risks (e.g., missing prerequisites, heavy credit load). 5. The Advisor reviews the AI-generated summary and visual schedule. 6. The Advisor selects "Approve Plan" or "Request

Task 3: Creating prototypes for app visualization in each use case.

Description: To provide a first, visualized look at how the app would work and to further explain the possible interactions when using the product.



2 Problem Statement

Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by: 24127543 – Nguyễn Trương Ngọc Thảo

Edited by:

Reviewed by:

2.1 Introduction

In order to graduate, students are required to finish a sequence of courses scattered throughout every semester over four years. During this time, the demand for clear instructions regarding semester planning, schedule clarity, and prerequisite chains is strict and requires an enormous amount of time.

Manually arranging schedules using scattered PDF files, various websites, disjointed catalogs, and spreadsheets is prone to human error. Some students cannot schedule the correct courses logically or track their academic results and remaining compulsory courses. This sometimes leads to delayed graduation timelines, which places an unnecessary financial burden on students and negatively impacts the university's graduation rate.

The objective of this software is to build a centralized platform for students to plan appropriate courses according to their current scores and remaining compulsory requirements, ultimately clearing a path to graduation. This solution will provide an interface where students can check their current semester's schedule, scores, and available courses. It will also utilize an LLM to arrange and manage everything, from checking academic history and calculating course results to recommending a new course plan.

2.2 Operating environment

The system will operate as a web-based application to ensure maximum accessibility for the student body and administrative staff.

- **Client Interface:** The application will be accessed via web browsers. The backend system will be based on Python and FastAPI. Frontend will mainly use React and Vite. Programming language will be Typescript.

- **Server Environment:** The application will be hosted on a standard cloud-based server environment capable of handling high access during registration weeks.
- **Future Enhancement:** The system shall be designed to support future secure, read-only integration with the university Registrar and Student Information Systems. In the current version, course and academic data may be prepared manually or imported through CSV/MySQL seed scripts by the development team.

2.3 Design & Implementation Constraints

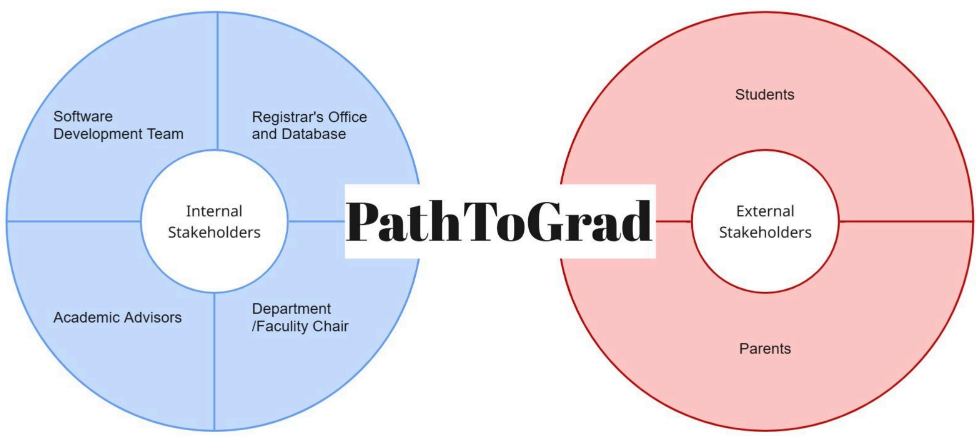
To ensure maintainability and consistency, the design and implementing progress must follow the listed constraints:

- **Technical:** The system will use Python for the backend to handle the complex scheduling logic, and MySQL to safely store course information and student records instead of json files.
- **Data:** Based on MySQL. The system's logic must follow the existing university rules. It can not allow students to register or generate a plan that conflicts with university's constraints, ensuring a clear schedule. The generated data will attempt to follow the official course data as close as possible.
- **LLM/AI:** OpenAI/Gemini API.
- **Document:** All source code must be documented according to advisors's software engineering practices.

3 Requirements Overview

3.1 Stakeholders

Written by: 24127543 - Nguyễn Trương Ngọc Thảo
Edited by:
Reviewed by:



No.	Stakeholder	Description
1	Software Development Team	Our development team works on the software and in the current version holds the data generating job.
2	Registrar's Office and Database	On future developments, the Registrar's Office holds the academic records and defines graduation rules.
3	Academic Advisors	The Academic Advisors can see the current class list, review student plans, records and identify possible conflict risks. Through this access, they can guide the student more precisely.

4	Department / Faculty Chairs	On future developments, the Department / Faculty Chairs are the people who dictate the curriculum, prerequisites, availability and decide changes on courses.
5	Students	Students are the main users of our software. They may plan from one semester, including checking its prerequisites, checking courses availability and schedule, to the full graduate path where they track graduation progress, current GPA, with notifications on academic risks.
6	Parents	While they have no access to our software, they are the guardians of students, which have a direct investment relation (tuition fee) due to the result generated by our system.

3.2 Requirements

3.2.1. Functional Requirements Specification

Written by: 24127563 - Trần Thị Ngọc Trâm

Edited by:

Reviewed by:

The functional requirements describe the main functions that PathToGrad must provide. These requirements are based on the proposal, the stakeholder needs, and the current system scope. The system focuses on academic planning, prerequisite checking, graduation progress tracking, and LLM-supported study plan generation.

FR-S1: Student Data Management

ID	Functional Requirement
FR-01	The system shall allow students to create and update their academic profile.
FR-02	The system shall allow students to enter completed courses, failed courses, grades, and credits.

FR-S2: LLM Planning Agent and Tool Interface

FR-03	The system shall allow students to send planning requests to the LLM Planning Agent.
FR-04	The system shall allow the LLM Planning Agent to call internal tools through APIs.
FR-05	The system shall provide an MCP-compatible tool interface with structured tool inputs and outputs.

FR-S3: Course Data and Academic Checking

FR-06	The system shall retrieve course data from MySQL.
FR-07	The system shall check whether a student satisfies prerequisite rules.
FR-08	The system shall calculate graduation progress.

FR-S4: Study Plan Generation and Explanation

FR-09	The system shall generate a recommended semester plan.
FR-10	The system shall detect academic risks in a study plan.
FR-11	The system shall explain the recommended plan in clear language.
FR-12	The system shall show warnings when a plan contains academic risks.
FR-13	The system shall provide fallback planning mode when the LLM provider is unavailable.
FR-14	The system shall generate template-based explanations in fallback mode.

FR-S5: Advisor, Admin, Traceability

FR-15	The system shall allow academic advisors to view plan summaries and warnings.
FR-16	The system shall allow academic staff or admin users to update course and prerequisite data.
FR-17	The system shall record agent runs, tool calls, provider errors, and fallback actions.

3.2.2. *Non-Functional Requirements Specification*

Written by: 24127543 - Nguyễn Trương Ngọc Thảo

Edited by:

Reviewed by: 24127563 - Trần Thị Ngọc Trâm

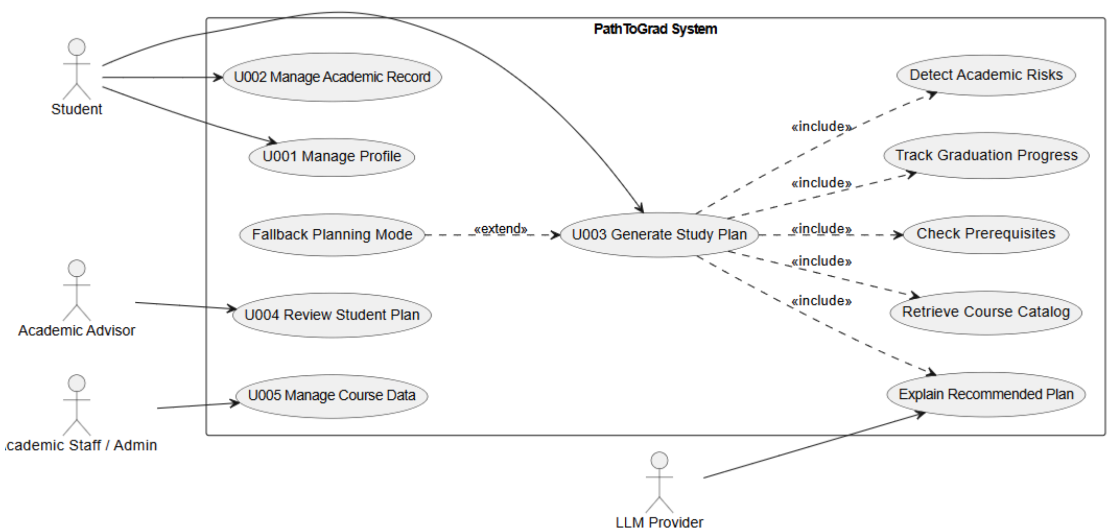
ID	Non-functional Requirement
NFR-01	The system should be accessible through modern web browsers.
NFR-02	The system should present planning results in clear and simple language.
NFR-03	The system should protect student academic information.
NFR-04	The system should restrict access based on user roles.
NFR-05	The system should store course, prerequisite, student, and academic record data in MySQL.
NFR-06	The system should verify LLM-generated recommendations using rule-based tools.
NFR-07	The system should continue core planning functions when the LLM provider is unavailable.
NFR-08	The system should show clear warnings when data is missing, outdated, or uncertain.
NFR-09	The system should log important agent actions and tool calls for traceability.

4 Requirements Analysis

4.1 Use Case model

Written by: 24127563 - Trần Thị Ngọc Trâm
Edited by:
Reviewed by:

The use case diagram describes the main interactions between PathToGrad and external actors. The Student can manage a profile, manage academic records, and generate a study plan. The Academic Advisor can review student plans, while Academic Staff/Admin can manage course data. The LLM Provider supports explanation generation when available. The central use case is U003 Generate Study Plan, which includes retrieving course data, checking prerequisites, tracking graduation progress, detecting academic risks, and explaining the recommended plan.



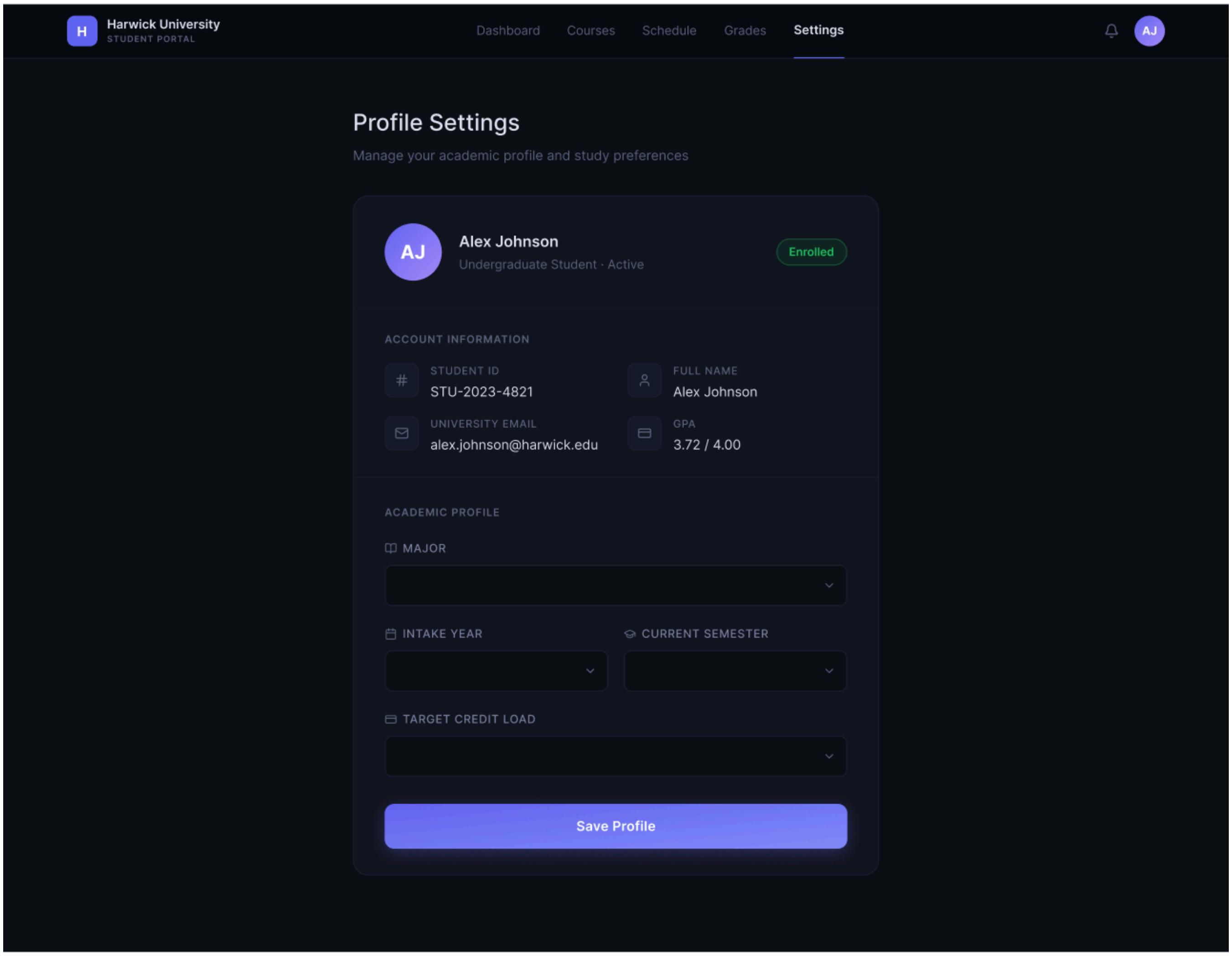
4.2 Use Case Specification

Use cases must be presented in order of their importance or according to the logical flow of the use case workflow.

4.2.1. *U001 Manage Profile*

Written by: 24127563 - Trần Thị Ngọc Trâm
Edited by:
Reviewed by:

Use case ID	U001
Use Case	Manage Profile.
Brief Description	This use case allows students to create and update their academic profile. The profile includes major, intake year, current semester, and target credit load. The system uses this information for course planning.
Actor	Student.
Pre-Condition	The student has opened the PathToGrad system.
Result	The student profile is saved and can be used by the LLM Planning Agent and internal planning tools.
Main Scenario	1. The student opens the profile page. 2. The system displays the profile form. 3. The student enters major, intake year, current semester, and target credit load. 4. The student submits the form. 5. The system validates the input data. 6. The system saves the student profile. 7. The system shows a success message.
Alternative Scenarios	**Missing required information:** The student leaves a required field empty → The system shows error messages → The student corrects the missing information and submits. **Invalid target credit load:** The student enters invalid credit load → The system shows error message → The student edits the value and submits.
Non-Functional Constraints	The system should present the profile form in clear and simple language. The system should protect student academic information. The system should restrict access to the correct user role.



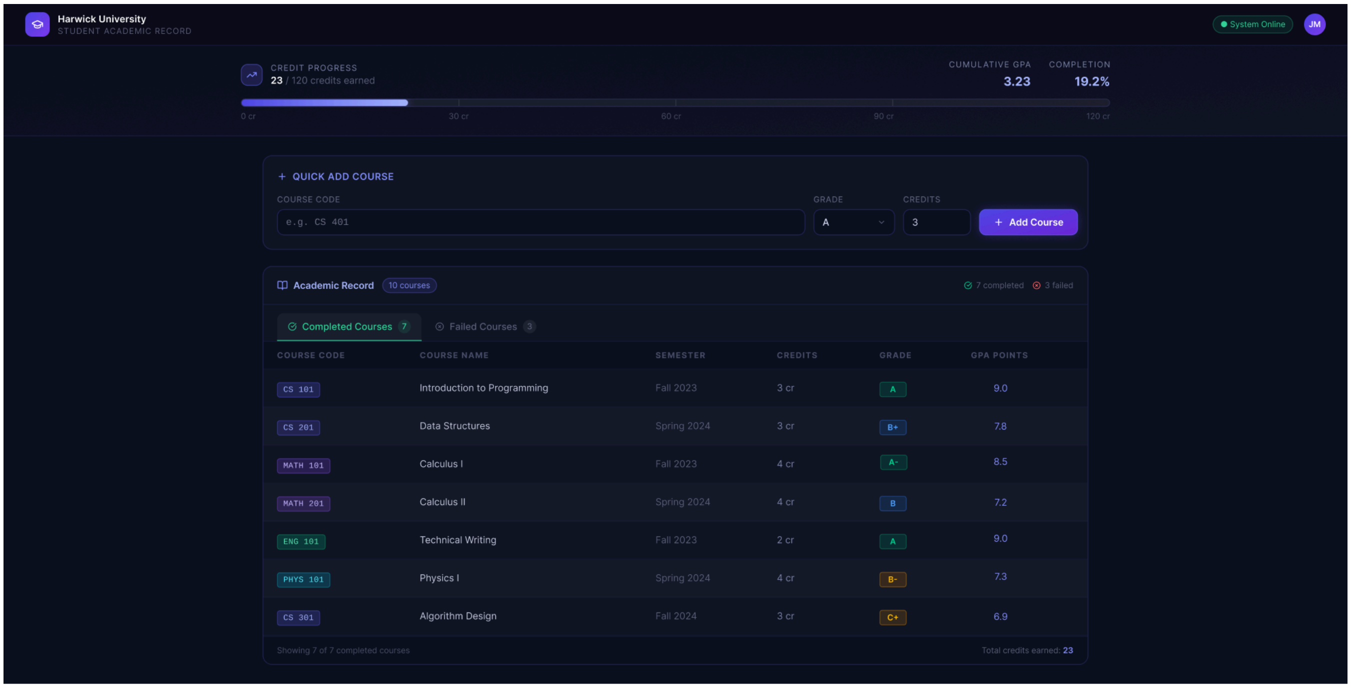
Visualized prototype of the Manager Profile(1)

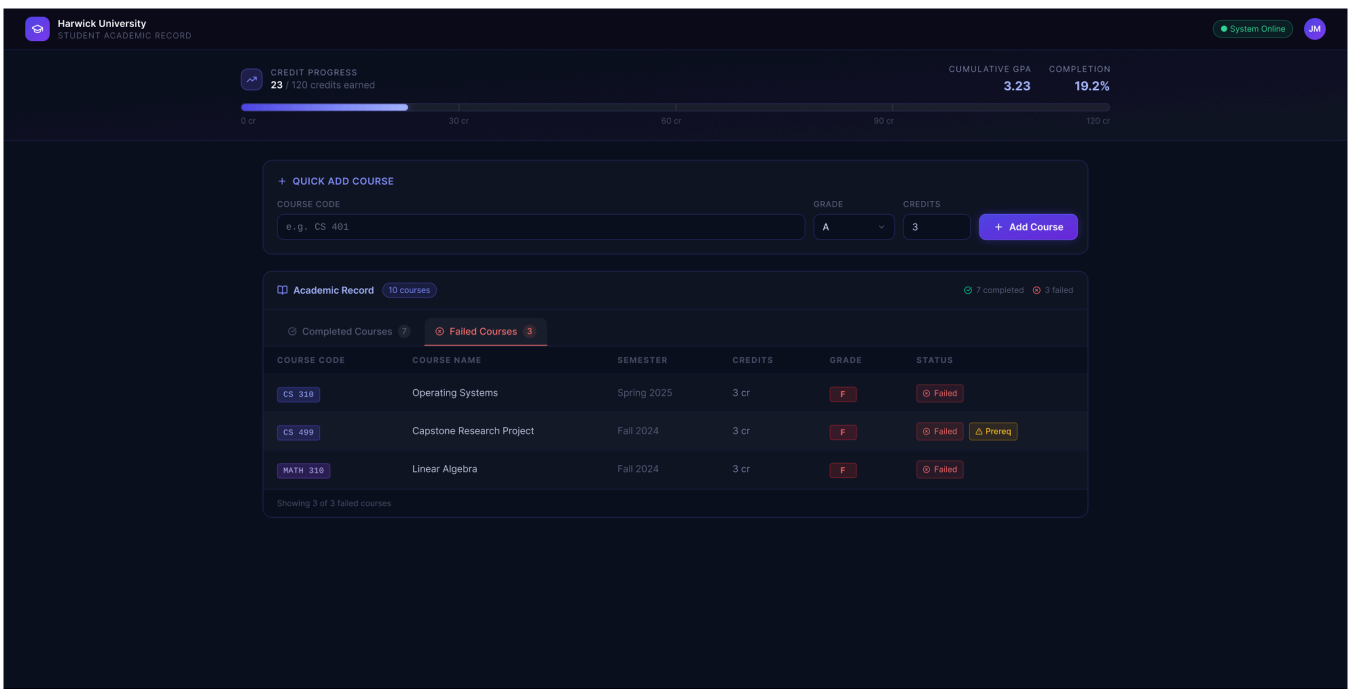
4.2.2. U002 Manage Academic Record

Written by: 24127563 - Trần Thị Ngọc Trâm
Edited by:
Reviewed by:

Use case ID	U002
Use Case	Manage Academic Record.
Brief Description	This use case allows students to enter and update academic records, including completed courses, failed courses, grades, and credits. The

	system uses this data for prerequisite checking and graduation progress tracking.
Actor	Student.
Pre-Condition	The student profile already exists in the system.
Result	The academic record is saved and can be used by the planning tools.
Main Scenario	1. The student opens the academic record page. 2. The system displays the academic record form or course list. 3. The student enters completed courses, failed courses, grades, and credits. 4. The student submits the academic record. 5. The system validates the entered data. 6. The system saves the academic record. 7. The system updates prerequisite status and graduation progress data.
Alternative Scenarios	**Course not found:** The student enters a course that does not exist in the course catalog → The system shows a warning message → The student selects a valid course from the available course list. **Missing grade or credit information:** The student submits a record with missing grade or credit information → The system asks the student to complete missing fields → The student updates and submits.
Non-Functional Constraints	• The system should store academic record data in MySQL. • The system should protect student academic information. • The system should show clear warnings when data is missing or uncertain.





Visualized prototype of the Manage Academic Records(2)

4.2.3. U003 Generate Study Plan

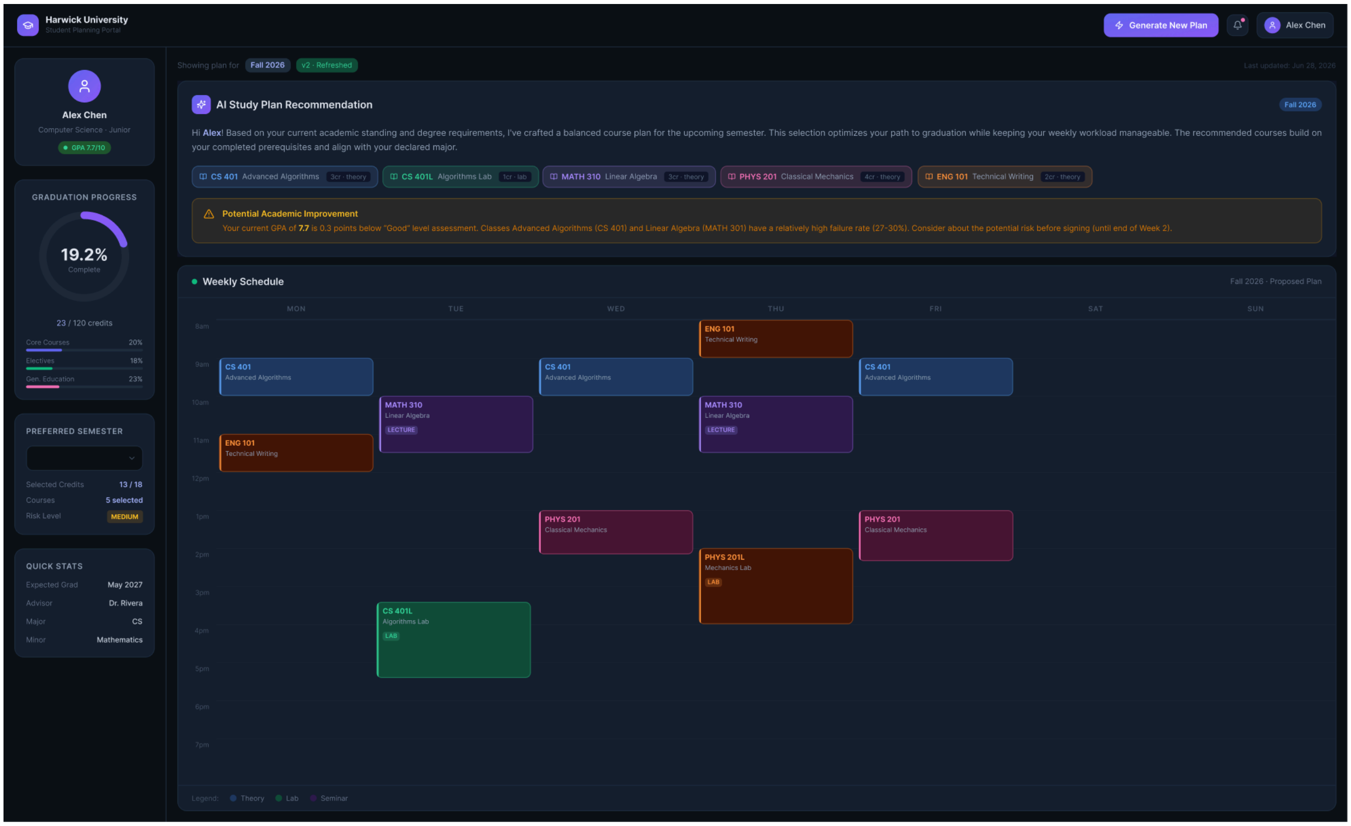
Written by: 24127563 - Trần Thị Ngọc Trâm

Edited by:

Reviewed by:

Use case ID	U003
Use Case	Generate Study Plan.
Brief Description	This use case allows students to request a recommended semester plan. The LLM Planning Agent receives the request, calls internal tools, checks academic rules, and returns a study plan with explanations and warnings.
Actor	Student & LLM Provider (Support).
Pre-Condition	The student profile, academic record, and course catalog data are available in the system.
Result	The system returns a recommended study plan with prerequisite status, graduation progress, academic risks, and explanation.
Main Scenario	1. The student opens the study planning page. 2. The student enters a planning request, such as target credit load or preferred semester. 3. The system sends the request to the LLM Planning Agent. 4. The LLM Planning Agent reads the student profile and academic record. 5. The agent calls the Course Catalog Tool to retrieve course data. 6. The agent calls the Prerequisite Checker Tool to remove invalid course options. 7. The agent calls the Graduation Progress Tracker to calculate completed and missing requirements. 8. The agent calls the Semester Plan Generator to create a recommended plan. 9. The agent calls the Academic Risk Detector to identify possible academic risks. 10. The system returns the recommended study plan. 11. The system explains why each course is recommended and shows warnings if needed.

Alternative Scenarios	**Missing academic record:** The student requests a study plan without entering academic records → The system shows a warning message → The student is asked to complete academic record information. **Missing or uncertain course data:** The system cannot find enough course or prerequisite data → The system shows warning that the recommendation is based on limited data → The system returns only the verified parts of the plan. **LLM provider unavailable:** The LLM provider fails or does not respond → The system switches to fallback planning mode → The rule-based tools generate a study plan → The system provides template-based explanations instead of AI-generated explanations.
Non-Functional Constraints	The system should verify LLM-generated recommendations using rule-based tools. The system should continue core planning functions when the LLM provider is unavailable. The system should log important agent actions and tool calls for traceability. The system should present planning results in clear and simple language.



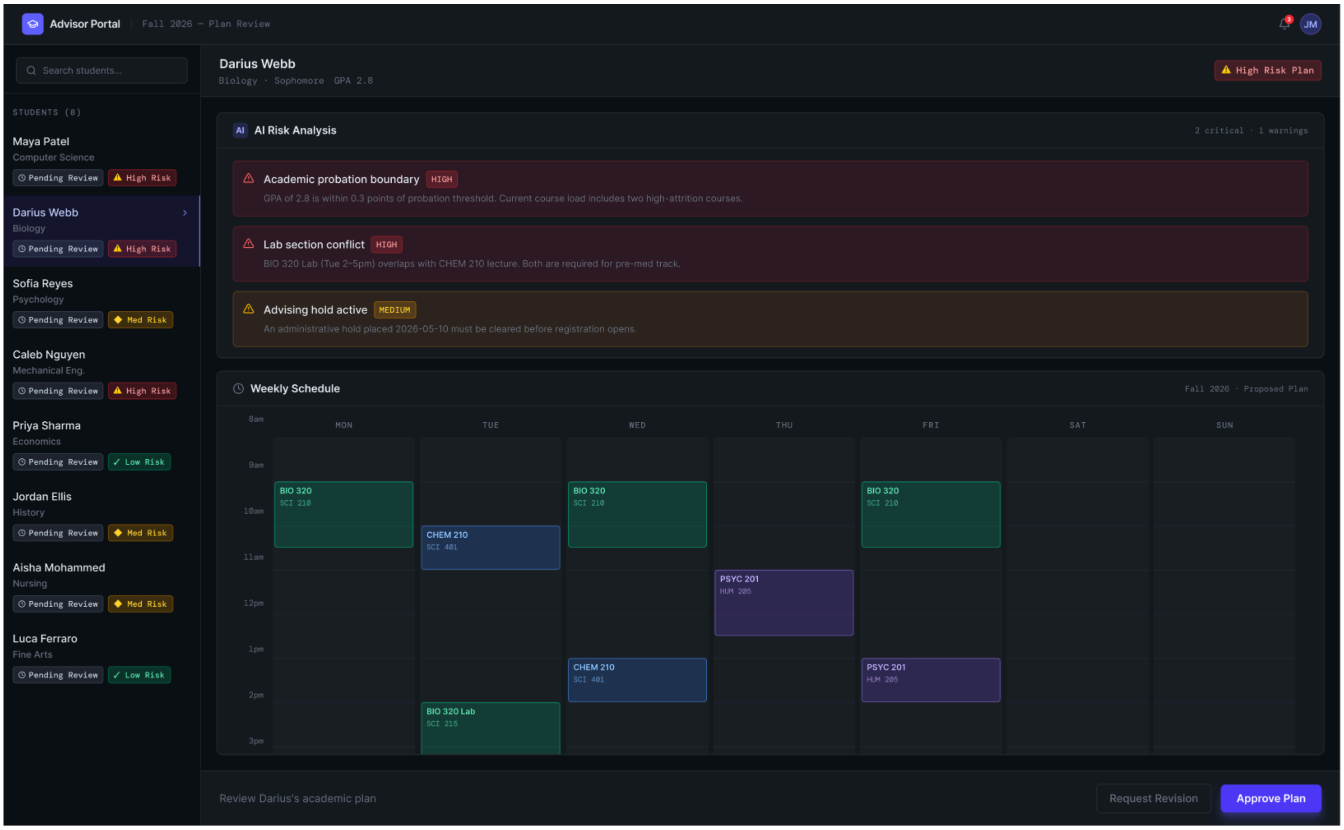
Visualized prototype of the Study Plan Generator(3)

4.2.4. U004 Advisor Review Specification

Written by: 24127080 - Ông Khánh Minh
Edited by:
Reviewed by:

Use case ID	U004
Use Case	Advisor Plan Review
Brief Description	Allows Academic Advisors to view a summarized dashboard of a student's LLM-generated semester plan, evaluate identified academic risks, and provide official feedback or approval.
Actor	Academic Advisor
Pre-Condition	The Advisor is authenticated in the system. The target student has successfully generated at least one semester plan using the LLM Planning Agent or Fallback Mode.
Result	The student's plan is marked as "Approved" or "Needs Revision" with attached advisor comments.
Main Scenario	1. The Advisor navigates to the "Student Plans Dashboard". 2. The system retrieves and displays a list of pending student plans. 3. The Advisor selects a specific student. 4. The system triggers the *Advisor Summary Skill*, fetching the student's academic profile, the proposed 7-day schedule, and any flagged academic risks (e.g., missing prerequisites, heavy credit load). 5. The Advisor reviews the AI-generated summary and visual schedule. 6. The Advisor selects "Approve Plan" or "Request Revision" and inputs an optional text comment. 7. The system updates the plan's status in the database and notifies the student.
Alternative Scenarios	Alt 1: LLM Summary Unavailable (Timeout): In step 4, if the API call to generate the summary fails, the system defaults to Fallback Mode, displaying the raw rule-based data and a template-based warning summary instead of the AI summary.

	Alt 2: Empty Data: In step 2, if no students have generated plans, the system displays a "No pending reviews" message.



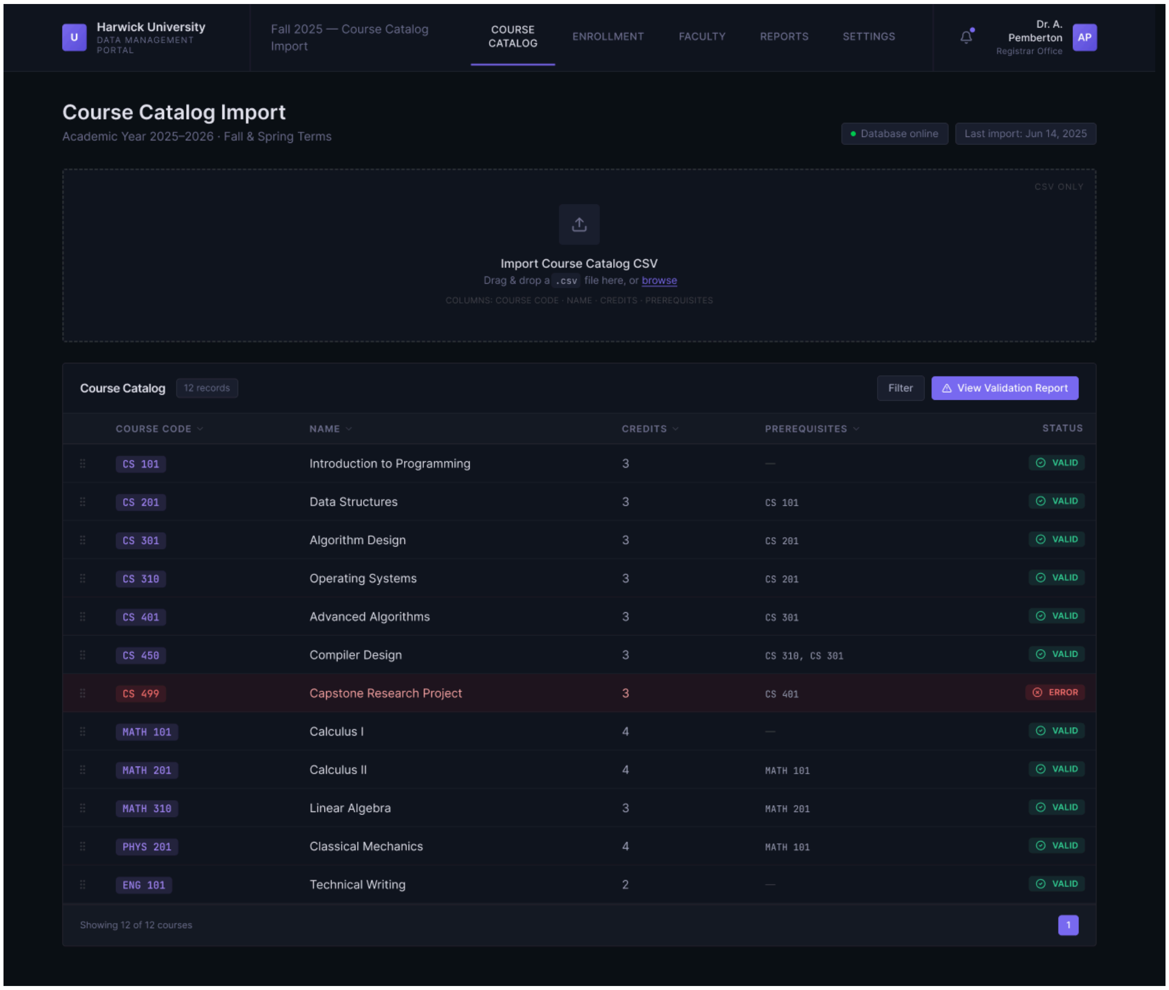
Visualized prototype of the Advisor Review(4)

4.2.5. U005 Course Data Management Specification

Written by: 24127080 - Ông Khánh Minh
Edited by:
Reviewed by:

Use case ID	U005
Use Case	Course Data & Prerequisite Management
Brief Description	Allows Academic Staff to maintain the university's course catalog by importing CSV files or using the Admin API to update course codes, credits, and prerequisite chains in the MySQL database.
Actor	Academic Staff / Admin
Pre-Condition	The Admin is authenticated with administrative privileges.
Result	The MySQL database is successfully updated with the latest course catalog and prerequisite rules, making them immediately available to the LLM Planning Agent.
Main Scenario	1. The Admin accesses the "Course Data Management" portal. 2. The Admin chooses to upload a new CSV file containing course metadata and prerequisite rules. 3. The system parses the file and executes a deterministic validation check (Core Logic Layer) to ensure there are no formatting errors or circular prerequisite loops. 4. The system displays a preview of the changes (e.g., "5 courses added, 2 prerequisites updated"). 5. The Admin clicks "Confirm Update". 6. The system commits the changes to the MySQL database and logs the import action.
Alternative Scenarios	Alt 1: Invalid Data Format or Logic Conflict: In step 3, if the system detects circular dependencies (e.g., Course A requires B, B requires A) or missing required fields, it halts the import, highlights the specific rows with errors, and prompts the Admin to fix them before proceeding. Alt 2: API Update: Instead of a CSV upload in step 2, the Admin pushes an update via the Admin API. The system performs the

	same validation (step 3) and returns a JSON success or error response.



Visualized prototype of the Course Management(5)

5 AI Usage Declaration

The prototype(s) included in each use case in **Section 4.2 - Use Case Specification** has AI usage for visualization aiding, listed as follows, and in order:

(1) **Figma**. *Claude Sonnet 4.6, on June 27th, 2026*, prompt: "A modern student profile settings page for a university web application. The layout features a clean, centered card with a form. The form includes labeled input fields and dropdowns for: Major, Intake Year, Current Semester, and Target Credit Load. Include a prominent 'Save Profile' primary button at the bottom. Keep the aesthetic minimalist and professional, perhaps utilizing a sleek dark theme, with a small, green 'Profile Saved Successfully' toast notification in the top right corner" used to assist section 4.2.1 of the report; AI generated sample prototype and design, student revised and adjusted the color scheme to match references.

(2) **Figma**. *Claude Sonnet 4.6, on June 27th, 2026*, prompt: "A student academic record dashboard for a university app. The top section features a horizontal 'Quick Add' form with input fields for Course Code, Grade, and Credits, next to an 'Add Course' button. Below the form is a wide, organized data table displaying two tabs: 'Completed Courses' and 'Failed Courses'. In the table, show a red warning tooltip next to one of the failed courses to indicate a missing prerequisite status. At the very top of the page, include a visual progress bar showing total accumulated credits." used to assist section 4.2.2 of the report; AI generated sample prototypes and design, student revised and adjusted the color scheme to match references.

(3) **Figma**. *Claude Sonnet 4.6, on June 27th, 2026*, prompt: "A student course planning interface for a university web app. The left sidebar shows a circular 'Graduation Progress' tracker and a small input field for 'Preferred Semester'. The main content area is split horizontally into two sections. The top half is an

'AI Study Plan Recommendation' card featuring a welcoming explanation of the suggested courses and a yellow warning banner highlighting a potential academic risk. The bottom half displays a 7-day visual weekly calendar schedule showing the recommended theory and lab class time slots blocked out. Include a 'Generate New Plan' button at the top" used to assist section 4.2.3 of the report; AI generated sample prototype and design, student revised and adjusted the color scheme and recalculated with data adjustments to match references.

(4) **Figma**. *Claude Sonnet 4.6, on June 27th, 2026*, prompt: "A modern web application dashboard for a university academic advisor. The left sidebar contains a vertical list of student names with colored status badges reading 'Pending Review'. The main content area is split horizontally into two sections. The top half is a clean summary card titled 'AI Risk Analysis' featuring three bullet points with warning icons. The bottom half displays a 7-day visual weekly calendar schedule showing blocked-out class time slots. At the bottom right of the screen, place a prominent primary button labeled 'Approve Plan' next to a secondary button labeled 'Request Revision'" used to assist section 4.2.4 of the report; AI generated sample prototype and design, student revised and adjusted the color scheme to match references.

(5) **Figma**. *Claude Sonnet 4.6, on June 27th, 2026*, prompt: "An administrative web portal for university data management. The top section features a prominent, dashed-border drag-and-drop file upload zone labeled 'Import Course Catalog CSV'. Below the upload zone is a wide data table with column headers for Course Code, Name, Credits, and Prerequisites. Overlaid in the center of the screen is a popup modal window titled 'Validation Report'. Inside the modal, show a mix of green success rows and a red error row highlighting a circular prerequisite conflict, with a final 'Commit to Database' button at the bottom of the modal." used to assist section 4.2.5 of the report; AI generated sample prototypes and design, students revised and adjusted the color scheme to match references.

6 Presentation

Teams are required to record presentation videos of their work using the provided template.

Every team member must participate in the presentation, specifically covering the sections their contributed to the project.

Videos must not exceed 30 minutes.

Please upload your videos to Youtube (set to either Unlisted or Public).

Provide the Youtube links to the field below.

7 Reflective Report

Which sections of this template are most helpful for implementation, and which are unnecessary? Provide specific examples to support your reasoning.